

NARWCDB Configuration Reference

This document describes every configuration file in `config/`, what each one controls, all current settings and their defaults, and known inconsistencies or stale items that need attention.

Overview

The config layer works as follows:

1. **load_config.m** is the primary config entry point. It merges defaults, local overrides, and an optional batch config and returns a plain struct. Use this for all new code.
2. **config/defaults/validation_config_default.m** (and `db_config_default.m`, `pipeline_config_default.m`) are the canonical source of truth for all defaults. Every tunable parameter must be defined here.
3. **config/local/db_config_local.m** (gitignored; created from the `.template`) holds per-machine overrides and database credentials.
4. **config/batches/<name>.m** (e.g. `migration.m`) hold per-run overrides that are applied on top of defaults and local config.
5. **format_definitions.json** defines the three recognized input file formats for the parser layer.
6. **get_config.m** is an older cached system still used by some non-validation code paths. Do not add new calls to it; prefer `load_config`.

Merge order

`defaults/* < config/local/db_config_local.m < config/batches/<name>.m`

Later layers win. Merge is deep (struct fields recursively merged).

Usage:

```
config = load_config()           % defaults + local
config = load_config('migration') % adds migration batch overrides

% SurveyValidator accepts a batch name directly:
validator = narwc.validation.SurveyValidator('migration');

% When calling rule functions directly in tests or scripts:
cfg = load_config(); cfg = cfg.validation;
narwc.validation.rules.coordinate_rules(data, collector, cfg);
```

Section: **paths**

All file and directory paths. Everything is derived from the location of `get_config.m` itself, so the project can be moved without changing any paths manually.

Field	Value	Notes
<code>paths.project_root</code>	Resolved absolute path of repo root	Computed via Java canonical path
<code>paths.src_dir</code>	<code><root>/src</code>	MATLAB source packages
<code>paths.data_dir</code>	<code><root>/data</code>	Data files
<code>paths.config_dir</code>	<code><root>/config</code>	This directory
<code>paths.tests_dir</code>	<code><root>/tests</code>	Test files
<code>paths.output_dir</code>	<code><root>/output</code>	Processed output (stale — see note below)
<code>paths.tables_dir</code>	<code><root>/data/tables</code>	Lookup table CSVs
<code>paths.raw_dir</code>	<code><root>/data/raw</code>	Incoming raw survey files
<code>paths.processed_dir</code>	<code><root>/data/processed</code>	Processed survey files
<code>paths.user_config</code>	<code><root>/config/user_config</code>	For user override file

Stale note: `paths.output_dir` points to `output/`, but `startup.m` creates `data/exports/` and `reports/` as the actual output destinations. The `output/` directory does not exist and is not created by startup. The processing section's `output_dir` inherits from `paths.processed_dir`, which is `data/processed/` — also not created by startup. This should be reconciled.

Lookup table `paths` Each entry maps a table name (lowercase) to its CSV file path under `data/tables/`. These are the names recognized by `get_lookup_table()`.

Key	CSV file
<code>behave</code>	<code>Behave.csv</code>
<code>beaufort</code>	<code>Beaufort.csv</code>
<code>speccode</code>	<code>SPECCODE.csv</code>
<code>taxcode</code>	<code>TAXCODE.csv</code>
<code>cloud</code>	<code>Cloud.csv</code>
<code>glare</code>	<code>GLARE.csv</code>
<code>wx</code>	<code>WX.csv</code>
<code>platform</code>	<code>PLATFORM.csv</code>
<code>contrib</code>	<code>Contrib.csv</code>
<code>idrel</code>	<code>IDREL.csv</code>
<code>confidnc</code>	<code>Confidnc.csv</code>
<code>strip</code>	<code>STRIP.csv</code>

Key	CSV file
photos	PHOTOS.csv
anhead	ANHEAD.csv
block	BLOCK.csv
ddsource	DDSOURCE.csv
idsource	IDSOURCE.csv
legstage	LEGSTAGE.csv
legtype	LEGTYPE.csv
stratum	STRATUM.csv

Missing entries: `foreign_key_rules.m` attempts to validate DTYPE, LEGGOOD, and OLDVIZ against lookup tables, but none of these three appear in `paths.lookup_tables`. `get_lookup_table()` will warn and return an empty table for them, silently skipping their validation. If these tables should be validated, add entries for `dtype`, `leggood`, and `oldviz` here.

Section: **validation**

Controls all MATLAB validation rule behavior. Consumed by `SurveyValidator` and individual rule functions.

Required fields

Field	Value
<code>validation.required_fields</code>	<code>['LAT_DD', 'LONG_DD', 'YEAR', 'MONTH', 'DAY']</code>

These five fields are checked for NULL/empty by `required_fields.m`. This is a minimal baseline; survey-type-specific required fields (FILEID, EVENTNO, TIME, BEAUFORT, etc.) are not yet added to this list.

Note: FILEID and EVENTNO are enforced as NOT NULL by the database schema directly, so they do not need to be in this list for the upload path. They may still be worth adding here for pre-upload reporting.

Behavioral validation

Field	Value	Notes
validation.behavioral.dead_behaviors	[0, 1, 2, 3]	Codes that indicate dead or stranded animal
validation.behavioral.active_swimming_codes	[6, 7, 8, 11–21]	Codes incompatible with dead behaviors
validation.behavioral.incompatible_behavior_pairs	[[6, 22], [22, 11]]	Pairs that cannot coexist in same sighting
validation.behavioral.call_codes	[40, 41, 42]	Presence of these without NUMCALF > 0 triggers a warning
validation.behavioral.taxcode_restrictions	(empty)	No taxcode-specific restrictions configured
validation.behavioral.species_restrictions	(empty)	No species-specific restrictions configured
validation.behavioral.behavior_data_tables		Removed redundantly; resolved from paths

Discrepancy: behavioral_rules.m reads config.taxcode_behavior_restrictions and config.species_behavior_restrictions, but get_config.m sets behavioral.taxcode_restrictions and behavioral.species_restrictions (without the behavior_infix). The rule function checks for the behavioral sub-struct and renames at extraction — verify this works end-to-end when restrictions are added.

Coordinate validation

Field	Value	Notes
validation.coordinates.lat_min	-90	Global hard minimum
validation.coordinates.lat_max	90	Global hard maximum
validation.coordinates.lon_min	-180	Global hard minimum
validation.coordinates.lon_max	180	Global hard maximum
validation.coordinates.study_lat_min	20	Warning if outside (also aliased as survey_lat_min)
validation.coordinates.study_lat_max	55	Warning if outside (also aliased as survey_lat_max)
validation.coordinates.study_lon_min	85	Warning if outside (also aliased as survey_lon_min)
validation.coordinates.study_lon_max	40	Warning if outside (also aliased as survey_lon_max)

Discrepancy vs. coordinate_rules.m defaults: The hardcoded default_config() inside coordinate_rules.m uses 35–50 °N and –75 to –60 °W. get_config.m

overrides this to 20–55 °N and –85 to –40 °W. The `get_config.m` values win at runtime. Neither matches the legacy SAS system’s hard cutoffs of 25–48 °N and 58–81 °W (i.e., –81 to –58 °W). The 20–55 / –85 to –40 bounds are likely intentionally wider to accommodate the full historical range, but this should be confirmed.

Date/time validation

Field	Value	Notes
<code>validation.datetime.year_min</code>	1970	YEAR below this is an error
<code>validation.datetime.year_max</code>	<code>year + 1</code>	YEAR above this is an error
<code>validation.datetime.year_warning</code>	1980	YEAR below this (but $\geq$ <code>year_min</code>) is a warning

Discrepancy vs. `datetime_rules.m` defaults: The hardcoded `default_config()` inside `datetime_rules.m` sets `year_warning = 1990`. `get_config.m` sets it to 1980. The `get_config.m` value wins at runtime. The legacy SAS system flagged years before 1990 as out of range (hard error), making 1990 the more conservative choice.

Species validation

Field	Value	Notes
<code>validation.species.require_valid_speccode</code>	1	Unknown SPECCODEs are errors
<code>validation.species.require_valid_taxcode</code>	1	Unknown TAXCODEs trigger a warning
<code>validation.species.speccode_table_speccode</code>	<code>data_dir/species/SPECCODE.csv</code>	Used for rules that read it directly
<code>validation.species.taxcode_table_taxcode</code>	<code>data_dir/species/TAXCODE.csv</code>	Used for rules that read it directly

Note: The `species_rules.m` function has additional configurable parameters with defaults in its own `default_config()`. These are not set in `validation_config_default.m` — the rule’s own defaults are used. To override them, add a `validation.species.*` field to `user_config.m` or a batch config.

Key species sub-config fields (all live in `validation.species.*`):

Field	Default	Notes
thresholds.group_size_default	10000	Fallback when neither SPECCODE nor TAXCODE provides a per-taxon threshold
thresholds.calf_count_default	100	Fallback calf count threshold
right_whale_max_group	50	Warning threshold for RIWH/NARW/SARW group size
right_whale_max_calves	5	Warning threshold for right whale calf count
require_speccode_for_sighting	true	SPECCODE required on sighting records
require_taxcode_for_sighting	true	TAXCODE required on sighting records
validate_speccode_lookup	true	Flag unknown SPECCODEs
validate_taxcode_lookup	true	Flag unknown TAXCODEs
validate_speccode_taxcode_mismatch	true	Flag TAXCODE mismatch against SPECCODE lookup
allow_numcalf_exceeds_half	false	When true, suppresses the warning that NUMCALF > NUMBER/2. Set to true in the migration batch config because small historical groups with one calf routinely exceed this ratio.

Environmental validation

Field	Value	Notes
validation.environmental.beaufort_codes	0:8	Valid beaufort codes (0–8 oktas)
validation.environmental.visibility_warning	50	VISIBILITY warning threshold (nautical miles)
validation.environmental.suppresses_negative_visibility	true	Suppresses negative visibility error — FIXME: legacy only
validation.environmental.surface_temp_warning	12	SURFACE_TEMP warning threshold (°C)
validation.environmental.surface_temp_warning	15	SURFACE_TEMP warning threshold (°C)
validation.environmental.reference	0:9	Referenced here but validation is done via FK lookup
validation.environmental.reference	0:12	Referenced here but beaufort_rules.m reads config.valid_values not this field

Known issue: beaufort_rules.m reads config.valid_values from its config sub-struct, but get_config.m sets validation.environmental.beaufort_values. When beaufort_rules.m is called with full_config.environmental as the config, it looks for config.valid_values which is not present — so it falls through to its own default_config() (which also returns 0:12). The

end result is correct, but the beaufort_values field in get_config.m has no actual effect.

Known issue: visibility_allow_negative = true is marked with a FIXME comment noting it is for legacy data only. New surveys should never have negative visibility. This should be set to false in user_config.m or toggled per-run once the legacy migration is complete.

Note on CLOUD: cloud_values = 0:8 implies codes 0 through 8 are valid. This conflicts with the legacy SAS system which treated only 0–4 and 9 as valid (9 being a “missing” sentinel). See docs/validation_reference.md §4 for the full discussion of this open issue.

Section: **processing**

Controls the data processing pipeline.

Field	Value	Notes
processing.output_dir	subdir/data/processed	Output destination for processed files
processing.default_steps	See below	Processing steps run by default
processing.duplicate_fields	{'LAT_DD', 'LON_DD', 'YEAR', 'MONTH', 'DAY', 'TIME', 'SPECCODE'}	Fields used for duplicate detection — stale, see note
processing.coordinate_precision	6	Decimal places for coordinate standardization
processing.outlier_std_threshold	Bier_std_threshold	Standard deviations for outlier flagging

Default processing steps:

```
remove_duplicates
standardize_coordinates
standardize_species_codes
flag_outliers
```

Stale field: processing.duplicate_fields references 'LON_DD', but the actual Master table column and MATLAB schema use 'LONG_DD'. This will silently fail to match any records and should be corrected to 'LONG_DD'.

Section: **database**

Default database connection parameters used when no `db_config.m` override is present.

Field	Value	Notes
database.server	'localhost'	SQL Server host
database.database	'NARWC'	Database name — inconsistent with schema
database.driver	'SQL Server'	ODBC driver name
database.timeout	30	Connection timeout (seconds)
database.trusted_connection	true	Windows integrated authentication
database.connection_string	Computed from above	ODBC connection string template

Critical inconsistency: `get_config.m` sets `database.database = 'NARWC'` but the actual database is named `NARWCDB` (as defined in all schema scripts and `db_config_template.m`). The `db_config_template.m` uses `DatabaseName = 'NARWCDB'`. Any code that connects using `get_config('database')` will attempt to connect to the wrong database name. This should be corrected to `'NARWCDB'`.

Separate credential system: `db_config_template.m` / `db_config.m` uses different field names (Type, Server, Port, DatabaseName, Username, Password, DataSource, Timeout) and is designed for `narwc.db.Connection`. `get_config.m`'s database section is a separate parallel config that may be consumed by other code paths. These two systems should be reconciled.

Section: **logging**

Controls console and file logging.

Field	Value
logging.level	'INFO' — valid levels: DEBUG, INFO, WARNING, ERROR
logging.console_output	true
logging.file_output	false
logging.log_file	'' — must be set if file_output is true
logging.timestamp_format	'%Y-%MM-%dd %HH:mm:ss'

config/local/db_config_local.m — Per-machine overrides

Template: config/local/db_config_local.m.template Actual file: config/local/db_config_local.m (gitignored; not in version control)

Create this file by copying the template. Any field set here is deep-merged on top of the defaults by load_config(). Only fields you want to change need to be included.

```
function config = db_config_local()
    % Override database connection for a local SQL Server instance
    config.db.server = 'MYPC\SQLEXPRESS';
    config.db.database = 'NARWCDB';
end
```

Validation and pipeline overrides that are specific to your machine (not to a batch run) can also go here:

```
% Tighten survey area bounds to match SAS system
config.validation.coordinates.survey_lat_min = 25;
config.validation.coordinates.survey_lat_max = 48;
config.validation.coordinates.survey_lon_min = -81;
config.validation.coordinates.survey_lon_max = -58;

% Fix legacy visibility flag after migration is complete
config.validation.environmental.visibility_allow_negative = false;
```

config/local/db_config_local.m.template — Database credential template

Actual file: config/local/db_config_local.m (gitignored; must be created before first use)

Used by narwc.db.Connection.create() to establish the database connection. Fields:

Field	Example value	Notes
Type	'SQLServer'	Database engine; use 'SQLServer' for SQL Server
Server	'localhost\SQLEXPRESS'	Server name or IP; include instance name if needed
Port	1433	SQL Server default; omit for named instances
DatabaseName	'NARWCDB'	Must match the actual database name
Username	'your_username'	Leave blank if using Windows auth (trusted connection)

Field	Example value	Notes
Password	'your_password'	Leave blank if using Windows auth
DataSource	'NARWC_DSN'	ODBC DSN name (alternative to direct connection params)
Timeout	10	Connection timeout in seconds

Note: The template defaults show Type = 'MySQL'. The actual database is SQL Server. Change this to 'SQLServer' when creating your db_config.m.

format_definitions.json — Input file format definitions

Defines the three recognized input file formats for the parser layer. Consumed by `narwc.io.parsers.ParserFactory` when auto-detecting the format of an incoming file.

standard

Property	Value
name	"Standard NARWC Format"
delimiter	"\t" (tab)
header_row	1
parser	StandardFormat
description	Tab-delimited with all 55 standard fields

legacy

Property	Value
name	"Legacy Format"
delimiter	"," (comma)
header_row	1
parser	LegacyFormat
description	Comma-delimited legacy database export

neaq

Property	Value
name	"NEAQ Format"

Property	Value
delimiter	"," (comma)
header_row	2
parser	NEAQFormat
description	New England Aquarium survey format
field_mapping	LAT_DD → "Latitude", LONG_DD → "Longitude", SPECCODE → "Species"

Note: The parsers named here (StandardFormat, LegacyFormat, NEAQFormat) must exist as classes in `src/+narwc/+io/+parsers/`. As of the current development phase, these classes may not all be fully implemented. See `CLAUDE.md` for the known-incomplete state of the parser layer.

get_lookup_table.m — Lookup table loader

Not a config file itself, but it is the runtime bridge between `paths.lookup_tables` and all validation rules that need lookup data.

Usage:

```
data = get_lookup_table('behave')    % Returns MATLAB table from Behave.csv
data = get_lookup_table('speccode')  % Returns MATLAB table from SPECCODE.csv
```

Behavior: - Resolves the file path through `get_config('paths').lookup_tables`
 - Returns an empty `table()` with a warning if the name is unknown or the file is missing
 - Reads all columns as strings initially, then attempts to convert the Value column to numeric if all values parse as numbers
 - Preserves original column names from the CSV header

Error handling: Warnings are issued rather than errors, so a missing lookup table causes silent validation skip. Check the MATLAB console for `get_lookup_table:FileNotFound` warnings if validation results seem incomplete.

reload_config.m — Cache-busting utility

```
config = reload_config()
```

Calls `clear get_config` to flush the persistent cache, then calls `get_config()` to rebuild from scratch. Use this after editing any config file. The new config is returned and also cached for the rest of the session.

Dead / stub files

validation_config.m

This file exists but immediately throws an error:

```
error("Unexpected call of validation config")
```

It is not called from anywhere in the codebase. It was an earlier, abandoned approach to configuration. The validation settings it contains are superseded by config/defaults/validation_config_default.m. This file can be removed.

Known issues summary

Issue	Location	Impact
database.database = 'NARWC' should be 'NARWCDB'	get_config line ~198	Connection to wrong database using the old get_config path
duplicate_fields contains 'LON_DD' instead of 'LONG_DD'	get_config line ~182	Duplicate detection silently misses all records
output_dir points to non-existent output/	get_config line ~68	Processing output goes to wrong or missing directory
visibility_allow_negative set in get_config.m no longer applies	get_config	Validation now reads from load_config() / validation_config_default.m; the default there is false (correct for new surveys)
year_warning = 1980 vs. old datetime_rules.m internal default of 1990	Resolved	validation_config_default.m is now authoritative; set to 1980
Survey area bounds (20–55 / –85 to –40) wider than SAS (25–48 / –81 to –58)	validation	More records pass the survey-area check than the SAS system would allow
db_config_local.m.template Type defaults to 'MySQL'	template file	New users may create config for wrong database engine; change to 'SQLServer'
DTYPE, LEGG00D, OLDVIZ missing from paths.lookup_tables	get_config	foreign_key_rules.m silently skips validation for these fields
validation_config.m is a dead stub	config/	Dead code; safe to delete

Issue	Location	Impact
db_config.m and get_config.m database sections use incompatible field names	Both	Connection code must know which config path it is using